Academic Project Report

On

Evaluation Metrics for POS Tagging Systems

Submitted By:

P. Thrivikram

Under the guidance of

Dr. Krishnendu Ghosh

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY

Evaluation Metrics for POS Tagging Systems

Author: Thrivikram Pudi
Project Type: Natural Language Processing – Comparative Evaluation Study
Tools & Frameworks: Python, NLTK, spaCy, Stanza, scikit-learn, HuggingFace Datasets
Repository: pos-tagging-evaluation

Abstract

Part-of-Speech (POS) tagging is a foundational task in Natural Language Processing (NLP) that assigns grammatical labels—such as noun, verb, adjective, or determiner—to each token in a sentence. The accuracy of POS tagging has a direct downstream impact on tasks including syntactic parsing, named entity recognition, and machine translation. This project presents a systematic comparative evaluation of four POS tagging systems spanning three paradigms: a hand-crafted rule-based baseline, a statistical tagger (NLTK), a neural pipeline tagger (spaCy), and a deep-learning-based tagger (Stanza). All models are evaluated on the `batterydata/pos_tagging` dataset hosted on HuggingFace, using Accuracy, Precision, Recall, F1 Score, and Confusion Matrix analysis as evaluation metrics. A key contribution of this work is the identification and resolution of a tokenization mismatch problem that arises when different NLP toolkits re-tokenize input sentences, which, if unaddressed, leads to artificially deflated performance scores. Through a manual token alignment strategy, we ensure that all models are compared on equal footing. Our results demonstrate that neural and deep-learning-based taggers (spaCy ≈ 0.95 F1; Stanza ≈ 0.94 F1) substantially outperform the rule-based baseline (≈ 0.22 F1), while the statistical NLTK tagger (≈ 0.93 F1) offers competitive performance with a simpler architecture.

1. Introduction

Natural Language Processing has witnessed remarkable advances over the past decade, progressing from hand-coded grammars to large-scale neural language models. At the heart of many NLP pipelines lies the seemingly simple but critically important task of Part-of-Speech tagging—the process of labeling every word in a sentence with its syntactic category. Despite its maturity, POS tagging remains an active area of research because of ongoing challenges such as ambiguity (e.g., the word "run" can be a noun or a verb), domain sensitivity, and the handling of unseen vocabulary.

The objective of this project is threefold:

Benchmark diverse paradigms. Compare the performance of rule-based, statistical, neural, and deep-learning POS taggers on a common, standard dataset to quantify how much modern architectures improve over traditional approaches.

Apply rigorous evaluation. Go beyond simple accuracy and employ Precision, Recall, F1 Score, and Confusion Matrix analysis to provide a multi-dimensional view of each model's strengths and weaknesses.

Address practical evaluation pitfalls. Identify and solve the tokenization mismatch problem that silently corrupts evaluation results when models internally re-tokenize input text, and demonstrate that careful token alignment is essential for fair comparison.

This report describes the dataset, methodology, implementation, results, and insights derived from the study.

2. Literature Survey

POS tagging has a long history in computational linguistics. Early systems relied on hand-crafted rules and morphological heuristics. Brill (1992) introduced a transformation-based learner that iteratively refined POS assignments using contextual rules, achieving accuracies that were competitive for its era but fundamentally limited by the coverage and specificity of the rule set.

The next generation of taggers adopted statistical methods. The Hidden Markov Model (HMM) tagger and the Maximum Entropy tagger leveraged annotated corpora—most notably the Penn Treebank (Marcus et al., 1993)—to learn transition and emission probabilities. NLTK's averaged perceptron tagger, which serves as one of the models evaluated in this project, falls into this category. These models demonstrated that data-driven approaches could significantly outperform manual rules, particularly for handling ambiguous tokens and unseen words.

The advent of deep learning brought further gains. Huang et al. (2015) introduced bidirectional LSTM-CRF models for sequence labeling, capturing long-range dependencies that statistical n-gram models could not. Frameworks such as spaCy (Honnibal & Montani, 2017) integrated convolutional and attention-based architectures into production-ready pipelines, offering both high accuracy and fast inference. Stanza (Qi et al., 2020), developed

by the Stanford NLP Group, employs a multi-layer BiLSTM architecture with character-level embeddings, achieving state-of-the-art results across multiple languages.

Recent work has explored transformer-based taggers using pre-trained models like BERT (Devlin et al., 2019) and RoBERTa (Liu et al., 2019). While these achieve marginal improvements on standard benchmarks, they come at substantially higher computational cost. In this project, we focus on the more widely deployed non-transformer models to provide a practical comparison relevant to resource-constrained settings.

A recurring but underreported challenge in POS tagging evaluation is tokenization consistency. Manning (2011) noted that differences in tokenization conventions between the gold-standard corpus and the system under evaluation can introduce systematic bias. This project directly addresses this gap by implementing explicit token alignment.

3. Dataset Description

The dataset used in this study is `batterydata/pos_tagging`, sourced from the HuggingFace Datasets library. It contains pre-tokenized English sentences annotated with Penn Treebank POS tags.

Key characteristics:

Property	Detail
Source	HuggingFace (`batterydata/pos_tagging`)
Language	English
Tagset	Penn Treebank (PTB) — 36 core tags + punctuation tags
Format	Each sample consists of a list of words and a corresponding list of POS labels
Split used	Train split
Preprocessing	Pre-tokenized; no additional cleaning required

The Penn Treebank tagset includes fine-grained distinctions such as VBD (verb, past tense), VBG (verb, gerund), NNP (proper noun, singular), and NNS (noun, plural), among others. This granularity makes the tagging task more demanding than coarser universal tagsets, but also more informative for downstream applications.

The dataset was loaded using the HuggingFace `datasets` library:

```
from datasets import load_dataset

def load_pos_dataset(split="train"):
    dataset = load_dataset("batterydata/pos_tagging")
    data = dataset[split]
    return data
```

4. Methodology

The experimental methodology follows a controlled pipeline designed to ensure fair comparison across fundamentally different tagging architectures.

4.1 Experimental Pipeline

Dataset Loading → Sentence Extraction → Per-Model Tagging → Token Alignment → Metric Computation → Visualization

Dataset Loading: The dataset is loaded from HuggingFace and the training split is selected.

Sentence Extraction: Each sample yields a list of gold-standard words and corresponding POS labels.

Per-Model Tagging: Each tagger receives the list of words and returns its predicted POS tags.

Token Alignment: For models that internally re-tokenize the input (spaCy, Stanza), a manual alignment step matches model-produced tokens back to the gold-standard tokens before comparison.

Metric Computation: Aligned predictions are evaluated against gold labels using Accuracy, weighted Precision, weighted Recall, and weighted F1 Score.

Visualization: A bar chart comparing model accuracies and a confusion matrix for the best-performing model are generated.

4.2 Token Alignment Strategy

The most critical methodological contribution is the handling of tokenization mismatches. When a NLP toolkit re-tokenizes the input sentence differently from the gold standard—for example, splitting a hyphenated word into multiple subtokens or merging contractions—the predicted tag sequence no longer aligns with the gold tag sequence. Naively comparing them index-by-index yields misleadingly poor results.

To resolve this, we implemented a two-pointer alignment algorithm that walks through both the gold-standard token list and the model's token list simultaneously:

If the gold token matches the model token at the current positions, the predicted tag is accepted and both pointers advance.

If there is a mismatch, the model pointer advances (skipping the model's extra sub-tokens) until a matching token is found.

Only matched tokens contribute to the evaluation.

This strategy ensures that misaligned tokens are filtered out rather than counted as errors, providing a fair assessment of the model's actual tagging ability.

5. Models Used

5.1 Baseline (Rule-Based)

The baseline tagger applies a small set of hand-crafted heuristic rules to assign POS tags:

Words in `{"the", "a", "an"}` → `DT` (Determiner)

Words ending in `"-ing"` → `VBG` (Verb, gerund)

Words ending in `"-ed"` → `VBD` (Verb, past tense)

Words starting with an uppercase letter → `NNP` (Proper noun)

All other words → `NN` (Noun, singular)

This tagger serves as a lower bound on performance. Its simplicity makes it incapable of handling ambiguity, context, or the full diversity of the PTB tagset.

5.2 NLTK (Statistical)

NLTK's default POS tagger is an averaged perceptron classifier trained on the Penn Treebank. It uses features derived from the current word, surrounding words, and word-internal properties (suffixes, capitalization) to make predictions. Unlike the rule-based baseline, it has been trained on a large annotated corpus, giving it strong coverage of English vocabulary and syntactic patterns.

```
import nltk

def nltk_pos_tag(words):
    tagged = nltk.pos_tag(words)
    return [tag for word, tag in tagged]
```

5.3 spaCy (Neural)

spaCy's `en_core_web_sm` model uses a convolutional neural network (CNN) architecture combined with a transition-based parser. The POS tagger component is trained end-to-end on OntoNotes 5.0 and uses hash embeddings and subword features to generalize to unseen vocabulary. Being neural, it captures contextual patterns that statistical models miss.

spaCy re-tokenizes input text internally, which necessitated the token alignment strategy described in Section 4.2:

```python
import spacy

nlp = spacy.load("en_core_web_sm")

def spacy_pos_tag(words):
    sentence = " ".join(words)
    doc = nlp(sentence)
    predicted_tags = []
    i, j = 0, 0
    while i < len(words) and j < len(doc):
        if words[i] == doc[j].text:
            predicted_tags.append(doc[j].tag_)
            i += 1
            j += 1
        else:
            j += 1
    return predicted_tags
```

5.4 Stanza (Deep Learning)

Stanza, developed by the Stanford NLP Group, employs a highway BiLSTM architecture with character-level and pre-trained word embeddings. It is designed for multilingual NLP and supports over 70 languages. Like spaCy, Stanza performs its own tokenization internally, requiring the same alignment procedure. The `xpos` field is used to extract Penn Treebank–compatible tags.

```python
import stanza

nlp = stanza.Pipeline(lang="en", processors='tokenize,pos', use_gpu=False)

def stanza_pos_tag(words):
    sentence = " ".join(words)
    doc = nlp(sentence)
    stanza_words, stanza_tags = [], []
    for sent in doc.sentences:
        for word in sent.words:
            stanza_words.append(word.text)
            stanza_tags.append(word.xpos)
    predicted_tags = []
    i, j = 0, 0
    while i < len(words) and j < len(stanza_words):
        if words[i] == stanza_words[j]:
            predicted_tags.append(stanza_tags[j])
            i += 1
            j += 1
        else:
            j += 1
    return predicted_tags
```

6. Implementation Details

The project is implemented in Python 3.10+ with a modular architecture:

Module	File	Responsibility
Dataset	`dataset/load_dataset.py`	Loads and returns the HuggingFace dataset
Taggers	`taggers/baseline_tagger.py`	Rule-based baseline
	`taggers/nltk_tagger.py`	NLTK averaged perceptron tagger
	`taggers/spacy_tagger.py`	spaCy CNN tagger + token alignment

	`taggers/stanza_tagger.py`	Stanza BiLSTM tagger + token alignment
Evaluation	`evaluation/metrics.py`	Precision, Recall, F1 (weighted, via scikit-learn)
	`evaluation/confusion_matrix.py`	Confusion matrix generation and plotting
Analysis	`analysis/visualization.py`	Bar chart for accuracy comparison
	`analysis/results_analysis.py`	Summary of findings
Orchestration	`main.py`	End-to-end pipeline execution

Key dependencies: `datasets>=2.0.0`, `nltk>=3.8`, `spacy>=3.7`, `stanza>=1.6`, `scikit-learn>=1.3`, `pandas>=2.0`, `matplotlib>=3.7`.

The evaluation loop in `main.py` iterates over every sentence in the dataset, invokes each tagger, performs token-level comparison (with alignment for spaCy and Stanza), and aggregates results across the entire corpus. Evaluation metrics are computed using scikit-learn's `precision_score`, `recall_score`, and `f1_score` functions with `average="weighted"` to account for class imbalance across the 36+ POS tags.

7. Evaluation Metrics

Five evaluation metrics were employed:

7.1 Accuracy

The proportion of tokens for which the predicted tag matches the gold tag:

Accuracy = Number of correctly tagged tokens/Total number of tokens

While intuitive, accuracy can be misleading when the tag distribution is highly skewed (e.g., nouns and determiners dominate English text).

7.2 Precision (Weighted)

Weighted precision measures, for each tag class, the fraction of tokens predicted as that class which are genuinely that class, then averages across classes weighted by their support:

Precision = TP / (TP + FP)

7.3 Recall (Weighted)

Weighted recall measures, for each tag class, the fraction of genuinely tagged tokens that the model successfully identified:

Recall = TP / (TP + FN)

7.4 F1 Score (Weighted)

The harmonic mean of precision and recall, providing a single balanced measure:

F1 = 2 * Precision * Recall / (Precision + Recall)

The weighted variant was used to handle the multi-class nature of the task.

7.5 Confusion Matrix

A per-class confusion matrix was generated for the best-performing model (spaCy) to visualize which tag classes are most frequently confused with each other, enabling qualitative error analysis beyond aggregate scores.

8. Results

8.1 Quantitative Results

Model	Paradigm	Accuracy	Precision	Recall	F1 Score
Baseline	Rule-based	0.3253	~0.23	~0.33	0.2252
NLTK	Statistical	0.9449	~0.94	~0.94	0.9384

spaCy	Neural (CNN)	**0.9533**	**~0.95**	**~0.95**	**0.9516**
Stanza	Deep Learning (BiLSTM)	0.9452	~0.94	~0.95	0.9439

8.2 Key Observations

spaCy achieved the highest scores across all metrics, with an accuracy of **0.9533** and an F1 score of **0.9516**, making it the most reliable model in this evaluation.

Stanza performed competitively (accuracy 0.9452, F1 0.9439) but was marginally behind spaCy, likely due to slight differences in token alignment coverage and the overhead of BiLSTM processing.

NLTK delivered strong results (accuracy 0.9449, F1 0.9384) despite its simpler statistical architecture, demonstrating that well-trained perceptron models remain highly competitive.

Baseline confirmed that rule-based heuristics are fundamentally inadequate for real-world POS tagging, with accuracy and F1 scores roughly 3× and 4× below the best model, respectively.

8.3 Impact of Token Alignment

The token alignment contribution is best illustrated by comparing results before and after alignment for spaCy and Stanza:

Model	Without Alignment	With Alignment	Improvement
spaCy	~0.65	0.9533	+0.30
Stanza	~0.70	0.9452	+0.25

These dramatic improvements demonstrate that tokenization mismatch, not tagging quality, was the primary source of error in the naive evaluation. This finding underscores the importance of preprocessing consistency in NLP evaluation.

9. Analysis & Discussion

9.1 Why Rule-Based Fails

The baseline tagger only recognizes five patterns, mapping the entire English vocabulary to just five POS tags (DT, VBG, VBD, NNP, NN). The Penn Treebank tagset contains 36+ tags, meaning the baseline is structurally incapable of predicting tags such as IN (preposition), RB (adverb), JJ (adjective), or MD (modal verb). Words like "quickly" (RB), "beautiful" (JJ), and "can" (MD) are all incorrectly tagged as NN. The low accuracy (~32%) actually exceeds random chance (which would yield roughly 1/36 ≈ 2.8%) only because nouns are the most frequent tag class in English.

9.2 Statistical vs. Neural

NLTK's averaged perceptron tagger uses a fixed feature window—typically the current word, the previous word, the next word, suffixes, and capitalization patterns. This captures local context effectively but cannot model long-range dependencies. In contrast, spaCy's CNN-based tagger and Stanza's BiLSTM tagger both have the capacity to integrate information from across the entire sentence. The marginal improvement of neural over statistical (0.95 vs. 0.94 F1) suggests that for POS tagging—a largely local phenomenon—the benefits of global context are real but modest.

9.3 spaCy vs. Stanza

Despite both being neural models, spaCy slightly outperformed Stanza. Possible explanations include:

Training data: spaCy's `en_core_web_sm` is trained on OntoNotes 5.0, which may have better coverage of the constructs present in the evaluation dataset.

Architecture efficiency: spaCy's CNN architecture processes the input in a feed-forward manner, whereas Stanza's BiLSTM introduces recurrent computation that may be more sensitive to tokenization artifacts.

Alignment coverage: The two-pointer alignment may discard slightly more tokens for Stanza than for spaCy, affecting the evaluated sample.

9.4 Confusion Matrix Insights

The confusion matrix for spaCy revealed that the most common confusions occur between:

NN (noun, singular) and **NNP (proper noun)** — capitalization-sensitive distinctions.

VBD (past tense) and **VBN (past participle)** — morphologically identical forms (e.g., "played").

IN (preposition) and **RB (adverb)** — syntactically ambiguous words (e.g., "around," "before").

These confusions reflect genuine linguistic ambiguity rather than model deficiency, and are consistent with known challenges in the POS tagging literature.

10. Challenges and Solutions

Challenge 1: Tokenization Mismatch

Problem: spaCy and Stanza apply their own tokenization when processing text. When the input sentence `"It's a well-known fact"` is passed to spaCy, it may tokenize it as `["It", "'s", "a", "well", "-", "known", "fact"]`, while the gold standard has `["It's", "a", "well-known", "fact"]`. Index-by-index comparison produces catastrophic misalignment, reporting accuracy as low as ~13% for spaCy (when forced tokenization was used) and ~65% under naïve sentence input.

Solution: A two-pointer alignment algorithm was implemented that traverses both token sequences simultaneously, matching tokens by string equality and skipping unmatched model sub-tokens. This raised spaCy's evaluated accuracy from ~0.65 to 0.9533 and Stanza's from ~0.70 to 0.9452—a difference of 25–30 percentage points.

Challenge 2: Heterogeneous Tag Sources

Problem: Different models report POS tags through different attributes. NLTK returns tags directly from `pos_tag()`, spaCy uses `token.tag_` for PTB tags (distinct from the universal `token.pos_`), and Stanza uses `word.xpos` (as opposed to `word.upos`).

Solution: Each tagger wrapper was carefully written to extract the Penn Treebank–compatible tag from the correct attribute, ensuring apples-to-apples comparison across all models.

Challenge 3: Evaluation Metric Design

Problem: Simple accuracy is insufficient for a multi-class labeling task with 36+ classes and significant class imbalance (nouns and determiners far outnumber interjections or list markers).

Solution: Weighted Precision, Recall, and F1 Score were adopted using scikit-learn's implementation with `average="weighted"`, which accounts for class frequency. Additionally, a confusion matrix was generated to provide per-class insight beyond aggregate statistics.

11. Innovation

This project contributes several noteworthy elements beyond a routine benchmarking exercise:

Manual Token Alignment Framework. The two-pointer alignment algorithm provides a reusable, model-agnostic solution for comparing any tagger whose tokenization diverges from the gold standard. Without this, evaluation results are unreliable—a fact demonstrated by the 25–30 percentage point accuracy drop observed when alignment was absent.

Multi-Paradigm Comparison on a Common Benchmark. By evaluating rule-based, statistical, neural, and deep-learning models under identical conditions—same dataset, same metrics, same alignment procedure—this project provides a controlled comparison that isolates the impact of the modeling paradigm.

Modular, Extensible Architecture. The project's codebase is organized into clearly separated modules for data loading, tagging, evaluation, and analysis, making it straightforward to add new models (e.g., a BERT-based tagger) or new metrics (e.g., per-class F1 or Cohen's Kappa) without modifying the core evaluation pipeline.

Transparent Documentation of Failure Modes. Rather than reporting only final results, this project documents the iterative debugging process—including the initial poor results for spaCy (~0.65) and Stanza (~0.70)—providing insight into the practical challenges of NLP evaluation that are often omitted from published work.

12. Conclusion

This project conducted a rigorous, multi-metric comparison of four POS tagging systems representing three distinct paradigms: rule-based, statistical, and neural/deep-learning approaches. The key findings are:

Neural models lead, but the margin is narrow. spaCy achieved the best overall performance (F1 = 0.9516), but NLTK's simpler statistical tagger was only about 1.3 percentage points behind (F1 = 0.9384). For applications where speed and simplicity are paramount, statistical taggers remain a viable choice.

Rule-based methods are insufficient. The baseline tagger's F1 of 0.2252 confirms that hand-crafted rules cannot capture the complexity of English morpho-syntax. This result motivates the universal adoption of data-driven approaches.

Tokenization alignment is non-negotiable. The most impactful finding of this project is that tokenization mismatches can degrade apparent model accuracy by 25–30 percentage points. Any fair evaluation of POS tagging (or any token-level task) must ensure that the model's tokenization is aligned with the gold standard before computing metrics.

Weighted metrics provide richer insight. By combining Accuracy, Precision, Recall, F1, and Confusion Matrix analysis, we obtained a multi-dimensional view of model performance that simple accuracy alone cannot provide.

Overall, spaCy emerged as the recommended model for English POS tagging in production settings, offering the best balance of accuracy, speed, and ease of integration.

13. Future Work

Several directions for future investigation are suggested by this work:

Transformer-Based Models. Evaluating BERT, RoBERTa, or DeBERTa-based taggers on the same dataset would establish whether transformer architectures yield meaningful gains over the CNN and BiLSTM models tested here, and at what computational cost.

Domain-Specific Evaluation. Testing all models on out-of-domain text—social media posts, biomedical abstracts, legal documents—would reveal how well each paradigm generalizes beyond standard English prose.

Runtime and Efficiency Analysis. Adding wall-clock timing and memory profiling to the evaluation pipeline would provide practical guidance for deployment decisions, particularly for latency-sensitive applications.

Cross-Lingual Extension. Since Stanza supports 70+ languages, extending this comparative framework to non-English languages (e.g., Hindi, German, Chinese) would test the universality of the paradigm-level findings.

Error Analysis by Linguistic Category. A deeper qualitative analysis—grouping errors by syntactic context (e.g., garden-path sentences, relative clauses)—would identify specific linguistic phenomena where current models struggle.

Ensemble Methods. Investigating whether combining predictions from multiple taggers (e.g., majority voting across NLTK, spaCy, and Stanza) yields accuracy improvements beyond any individual model.

References

Systematic Review of POS Tagging (2022):
https://link.springer.com/article/10.1186/s40537-022-00561-y
Comparative Analysis of ML vs DL POS Tagging (2025):
https://www.sciencedirect.com/science/article/pii/S0957417425016471
BiLSTM POS Tagging (2023):
https://www.sciencedirect.com/science/article/pii/S1319157822004207

Survey of POS Tagging (2024):
https://www.researchgate.net/publication/379517712_A_Survey_of_Part-of-Speech_Tagging
NLTK vs spaCy Comparison (2024):
https://www.joaasr.com/index.php/joaasr/article/view/935
HuggingFace Dataset: “batterydata/pos_tagging” -
https://huggingface.co/datasets/batterydata/pos_tagging
NLTK Documentation: https://www.nltk.org/
spaCy Documentation: https://spacy.io/
Stanza Documentation: https://stanfordnlp.github.io/stanza/